
Data Engineering: Ingestion, Transformation and Lakehouse Design



Dr. Abdelkarim Erradi
CSE@QU

With assistance from AI tools, particularly
Microsoft Copilot and Google Gemini

Outline

1



Data Lakehouse

Source systems → Ingested data → governed, reliable, analysis-ready data

2



Data Pipeline Design Decisions

Batch vs. streaming • Push vs. pull • Full vs. incremental/CDC

3



Medallion Architecture

Bronze → Silver → Gold layers

4



Data Processing using SQL

Querying, joins, aggregation, and writes

5



Data Processing using DataFrame

Clean, standardize, validate, and enrich



Data Lakehouse

The Data Engineering Pipeline

Data engineering is the disciplined movement of data from messy sources to trusted, analysis-ready tables.



Pipeline ground rules:

Store raw data, then transform

- Load source data as received; clean it in a later step.
- A raw copy lets you rebuild without rereading the source.

Test every step, not the end

Check row counts, nulls, and unique keys after each transformation, and stop the pipeline when a check fails.

Design for reruns

Make jobs idempotent: replaying a day overwrites its rows rather than duplicating them.

OLTP Records Transactions vs. OLAP Answers Questions

	 OLTP - Operational Online Transaction Processing	 OLAP - Analytical Online Analytical Processing
Purpose:	Record day-to-day transactions.	Find patterns and answer business questions.
Typical action:	Insert or update a few rows.	Scan and summarize millions of rows
Optimized for:	Fast, reliable writes with low latency.	Read-heavy including joins, trends, and aggregations.
Example:	Place an order and update stock.	Compare monthly sales by region.

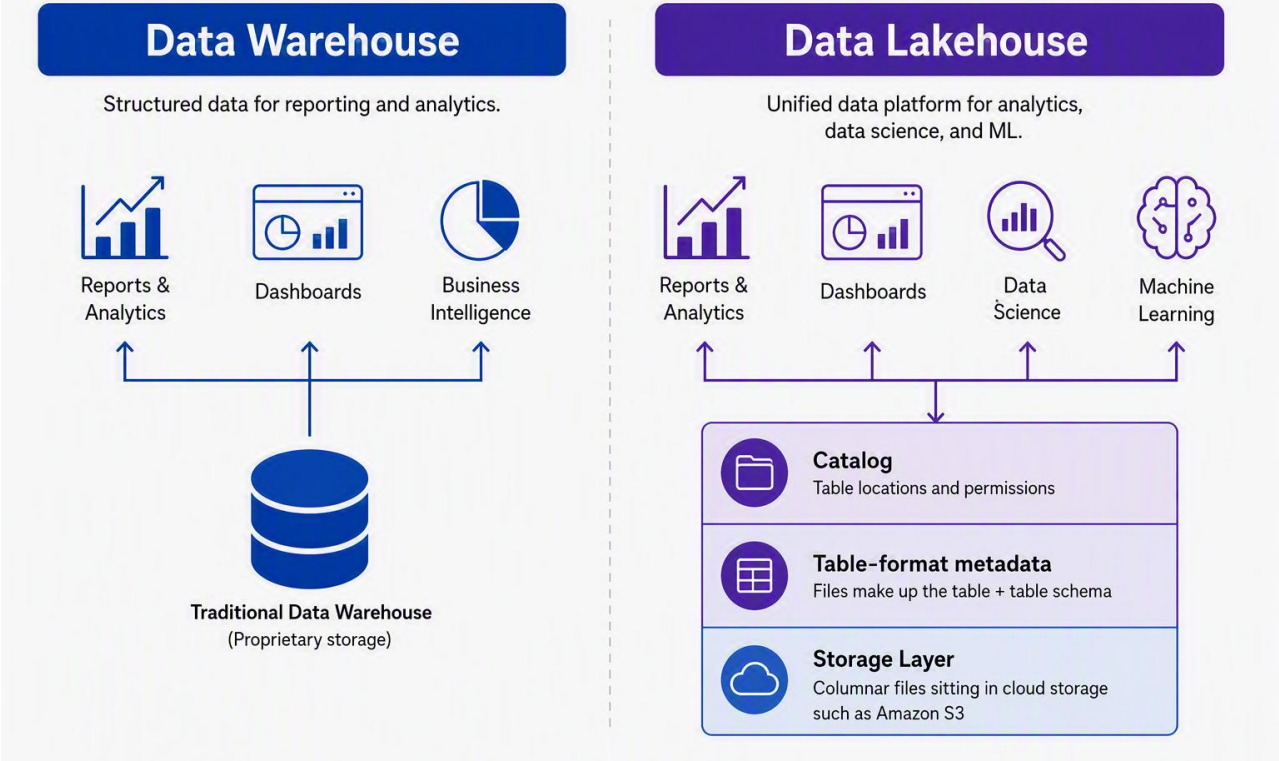
Why separate them?

Prevent analytical queries (full scans, joins, aggregations) from slowing down operational transactions, and vice versa



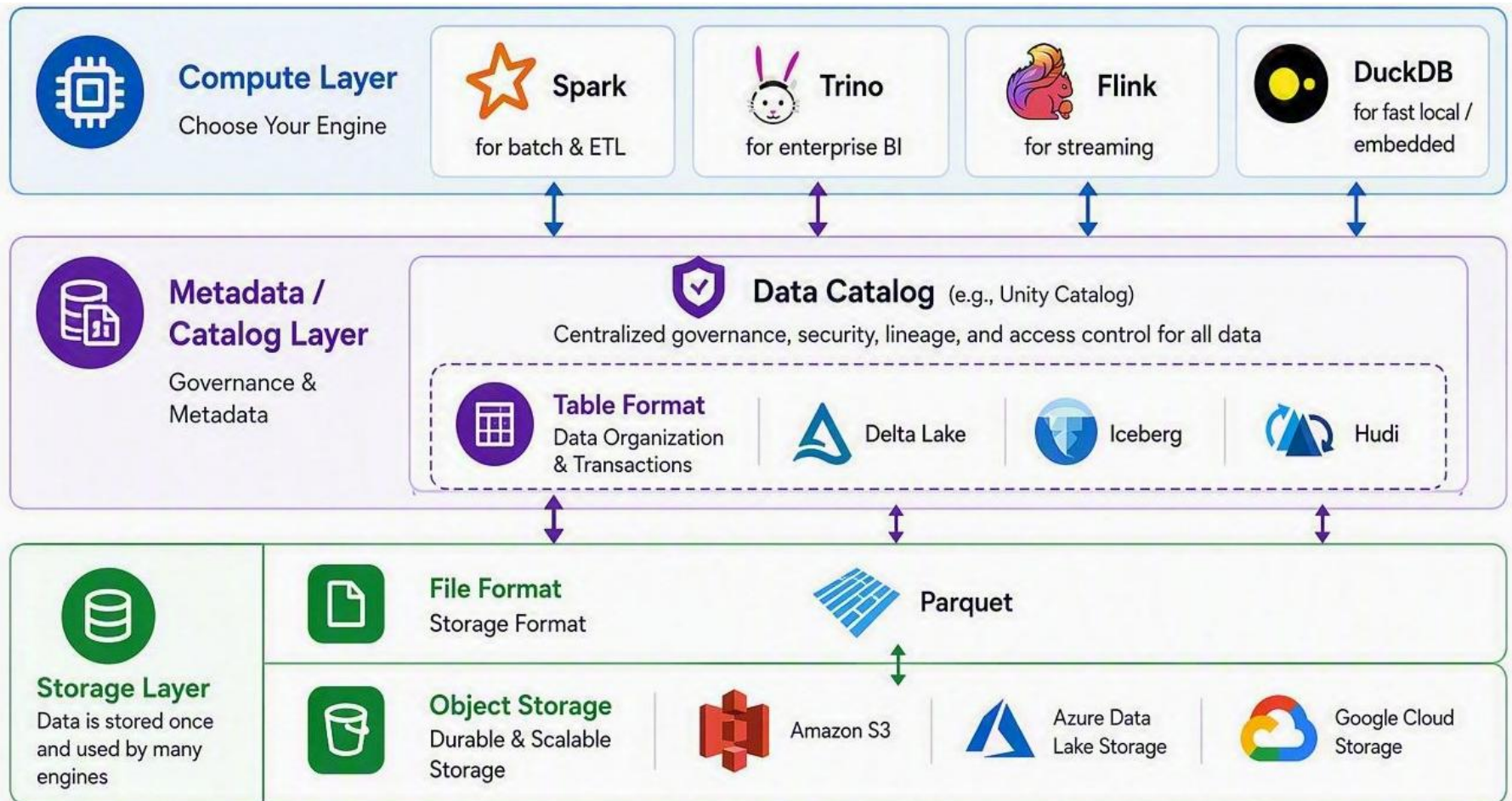
The business relies on both workloads: one supports daily operations, while the other enables analysis and insight

Data Platforms: Data warehouse vs. data Lakehouse



Lakehouse Architecture

A Lakehouse separates -here data lives (**storage**), how it is processed (**compute**), and how it is discovered and governed (**catalog**). Each layer scales independently.



Two Ideas Behind the Modern Lakehouse

	Storage and Compute Are Separate	Open Table Formats
What is it	Data sits in cheap cloud storage; engines connect to it when needed.	A metadata layer that turns plain files into proper tables .
Why it helps	Add or remove processing power without copying or reloading data.	Safe updates, deletes and schema changes, plus a view of older versions (time travel).
In practice	Any engine reads the same data, then shuts down when it is no longer needed.	Roll back a bad load, or reproduce yesterday's numbers.
Watch out	How the files are organized decides how fast queries run.	Tables need housekeeping: merge small files, clear old versions.
Together	Cheap storage keeps one copy of the data; open tables make changing it safe, so any engine can read the same trusted tables without copying them.	

How a Lakehouse Works



Compute Layer

Engines that read and process the data

Start one for a job and shut it down when done; multiple engines can read the same data simultaneously.

For example: Spark, Trino, Flink or DuckDB



Catalog & Metadata

Catalog

Keeps track of where tables are and how to find them.

Manage who can access the tables.

Table-format metadata

Tells data engines which files make up the table + table schema



Storage Layer

One shared copy of the data

Columnar files sitting in cloud storage — cheap, and charged separately from processing.

For example: Parquet files in S3, Azure Data Lake or Google Cloud Storage

How query runs:

Engine asks catalog → Catalog resolves table → Metadata selects snapshot files → Engine reads listed files → Result returned

Multi-engine flexibility: Add or swap an engine anytime, the data itself never moves.

Four Lakehouse Choices for Analytical Workloads



	Databricks	Snowflake	Microsoft Fabric	MotherDuck
What it is	Managed lakehouse on Spark for ETL, SQL, streaming, and ML	Managed SQL platform; compute and storage scale separately	Microsoft analytics suite tied to Power BI	Serverless DuckDB in the cloud, built on the open DuckLake format
Distinctive	One engine for batch, streaming, and ML on the same tables	Instant compute scaling with almost no tuning	One stack from ingestion to Power BI report	DuckLake keeps table metadata in a plain SQL database and data in Parquet
You pay for	Compute units + your cloud storage	Credits for compute + storage	Reserved capacity + storage	Free tier; paid plan + per-second compute and storage
Best for	Large-scale data engineering and AI/ML	Fast SQL analytics with minimal management	One integrated platform + Power BI	Small-to-mid data and fast, interactive analytics

Governance keeps data secure, findable and traceable

Capability	Practical question	Online-shop example
Data cataloging	What is it, who owns it and when is it refreshed?	The daily_product_sales table catalog entry includes: Description: daily sales totals for each product. Owner: Sales Analytics Team. Refresh: Every day at 6:00 AM.
Access control	Who can view, change or share it?	Analysts see sales; only Finance can access payment details.
Data lineage	Where did it come from and what transformed it?	Orders API → Bronze → Silver orders → daily_product_sales.
Monitoring and auditing	Is it healthy, and who used or changed it and when?	Alert on late loads; log access and permission changes.

Good governance helps people find the right data, trust it, and use it safely.



Data Pipeline Design Decisions

Pipeline Design: Five Decisions Before You Build

Agree with the source owner	What to decide	What we agreed (online orders)
1 Contact and support	Who owns the source; who resolves incidents	Commerce team owns sales data
2 Grain, schema, and key	What a single row represents (grain), the schema of each entity, and the key that uniquely identifies it	One row = one item in an order; key = order_id + line_id; 12 agreed fields
3 Change behavior	Append-only, or do records get updated and deleted?	Not append-only: orders can be updated or cancelled
4 Delivery and ingestion frequency	How often data is delivered and how fresh it must be	Hourly incremental file. Report delay up to 1 hour is acceptable
5 Data quality and recovery	What to do with bad rows; how to replay a failed run	Orders with no ID go to quarantine; source resends on request

Granularity: define what one row represents

Decision	Meaning and when to use it	Online-orders example
Detailed grain	Smallest business event Use for drill-down and root-cause analysis Define before choosing keys	One row per order item Key = (order_id, line_id) Basket-level analysis
Summary grain	Calculated group Use for KPIs, dashboards, trends, and reporting Derived from detailed data	One row per day Fields = date, sales_amount, order_count, avg_order_value
Best practice	One grain per table Store detail first; derive summaries Aggregation removes detail	Order items → Daily sales → Monthly sales

Processing mode: Batch or streaming - choose by latency

Ask: How quickly must the result update?

Compare	Batch processing	Stream processing
Execution	Scheduled or on demand; process and finish	Continuous or frequent micro-batches
Typical latency	Minutes to hours	Milliseconds to minutes
Order example	At 02:00, calculate yesterday's sales	Update a fraud score as payment events arrive
Use it when	Decisions can wait; simpler operations matter	Immediate action matters
Engineer for	Load watermark, retry, and idempotent rerun	Event time, replay, and deduplication

Decision rule: end-of-day report → batch · fraud alert before authorization → streaming

Pull vs. push: Who starts data delivery?

Ask: Does the consumer request data, or does the source send it?

Compare	Pull: consumer requests	Push: source sends
How it starts	The consumer checks for or reads new data.	The source or service sends data when it is ready.
Think of it as	"Do you have anything new?"	"I have something new for you."
Common patterns	API or database polling · Kafka consumer reads	Webhook · file arrival · managed message delivery
Real examples	Batch: nightly job reads changed orders Streaming: Kafka consumer reads events	Batch: supplier uploads a daily file Streaming: payment service sends a webhook
Best when	The consumer should control pace or replay.	Fast delivery matters and the source supports it.
Watch for	Polling too often, source load, and the saved read position.	Authentication, retries, and duplicate events.

Key idea: Pull or push is about direction—not speed. Either can be batch or streaming.

How much to load: Full, incremental or CDC

Ask: Do we reload everything, or only what changed?

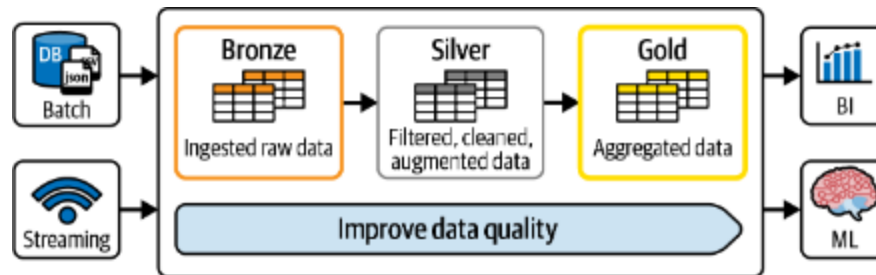
Option	What it means	Use it when	Online-order example
Full load	Copy the entire table every run	First load; or table is small	Reload full product catalogue nightly
Incremental load	Copy only new / changed rows since last run	Table too large to reload; Needs a reliable watermark (date or ID)	Load only orders placed since last night's run
Change Data Capture (CDC)	Stream every insert, update, and delete	Near-real-time sync required	Follow an order: placed → paid → cancelled

**Start with a full load, then process incremental changes periodically.
Use CDC only when downstream must stay in sync in near real time.**

Always record what was loaded and when.



Medallion Architecture



Medallion Architecture



- A Medallion architecture arranges data into three layers, enhancing the data's structure and quality as it progresses through the layers

Bronze → Silver → Gold: data gets cleaner at each stage

 Bronze	 Silver	 Gold
• Store exactly as received — never edit • Add tracking metadata: <code>source_name</code> · <code>ingested_at</code> · <code>load_id</code> • E.g., Raw order files from each store	• Fix types · handle nulls · remove duplicates • Standardize values · apply business rules • Enrich rows with reference data joins • E.g., One clean orders table, one row per order line	• Aggregate to the right grain · derive business metrics • Model facts and dimensions · rename for the business • One definition per metric built only on Silver data • E.g., Daily sales by product, ready for the dashboard
 <i>If processing fails, replay from Bronze — without asking the source to resend</i>	 <i>Never delete a bad row silently -> quarantine, or flag it</i>	 <i>Rebuild, never patch — check totals every run</i>

3 Bronze: the row data plus tracking columns

Ingestion details added to every row such as:	Online-order example	Why it matters
source_name	orders_api	Says which system the row came from
ingested_at	2026-09-03 10:03	Shows when it arrived -> can help identify late data
load_id	batch_2026_09_03_01	Lets you replay just one bad run

Write Bronze once, never edit it. Fix data in Silver, so you can always rebuild.



Silver Layer Data Transformations

16

Transformation	What it means	Online-order example
Fix data types	Text becomes numbers, dates and Booleans	"19.99" becomes 19.99; "2026-09-03" becomes a date
Handle missing values	Fill, flag, or quarantine blanks	Order with no customer_id goes to quarantine
Remove duplicates	Keep one row per unique key	Same order sent twice by the API: keep the latest
Standardize values	One spelling, one unit, one code, one format	"QA", "Qatar", "qatar" all become QAT; dates stored as YYYY-MM-DD
Apply business rules	Reject rows that cannot be true	quantity <= 0 or a future order_date is rejected
Integrate reference data	Connect each row to its related data	Order joined to its customer; product_id 88 gains its name
Enrich with new fields	Add a useful value calculated from existing fields	delivery_days = delivered_at – order_date

**Silver is where messy data becomes trusted, analytics-ready data.
Not every dataset requires every transformation step.**

See more details
in slides 03 *Silver
Transformations*



Gold Layer Data Transformations

18

Transformation	What it means	Online-order example
Aggregate to a grain	Roll many rows up to the level the business asks about	Aggregate individual order lines into units sold and revenue per product, per store, per day.
Derive business metrics	Turn raw columns into the agreed measures	$\text{revenue} = \text{quantity} \times \text{unit_price} - \text{discount}$
Model facts and dimensions	One table of events joined to descriptive tables	fact_sales joined to dim_product, dim_store and dim_date
Add time comparisons	Pre-compute the periods people always ask for	Month-to-date revenue and the same day last year
Rename for the business	Technical names become words users recognise	qty_ord becomes Units sold; amt_net becomes Revenue
Filter to the audience	Publish only the rows and columns a user may see	A store manager sees only their own store's rows

Gold is where trusted data becomes business-ready insights that people can act on.

See more details
in slides 04 Gold
Transformations



Data Processing using SQL



SQL — The Working Language of Data Engineering

SQL is how data engineers talk to data. It is a declarative language that enables you to describe **WHAT** results you want, and the query engine works out how to produce it.
Its commands fall into four families:

Family	Commands you will meet	What it is for
DDL Data Definition Language	CREATE, ALTER, DROP	Define the structure: tables, columns, schemas
DML Data Manipulation Language	SELECT, INSERT, UPDATE, DELETE	Read the data and change what is stored in it
DCL Data Control Language	GRANT, REVOKE	Control who may read or change each table
TCL Transaction Control Language	COMMIT, ROLLBACK	Group changes so they all succeed, or none do

```
-- DDL: define the table once
CREATE TABLE silver.orders (...);
-- DML: Load Bronze into Silver every day
INSERT INTO silver.orders
  SELECT * FROM bronze.orders;
-- DCL + TCL: grant access, commit safely
GRANT SELECT ON silver.orders TO analyst;
```



SELECT — One Statement, Four Jobs

	What it does	Example
SELECT	Choose or compute the columns returned	SELECT order_id, total * 1.05 AS with_tax
FROM	Name the table, or joined tables, the rows come from	FROM sales.orders
WHERE	Keep only the rows that qualify	WHERE status = 'shipped'
GROUP BY	Collapse many rows into one per group	GROUP BY customer_id
ORDER BY / LIMIT	Sort the result and cap how much comes back	ORDER BY revenue DESC LIMIT 10

```
-- Top 5 customers by September revenue
SELECT customer_id,
       SUM(order_total) AS revenue
FROM   sales.orders
WHERE  order_date >= '2026-09-01'
GROUP BY customer_id
ORDER BY revenue DESC LIMIT 5;
```



Key idea

- One statement answers four questions: which rows, which columns, grouped how, sorted how.



SELECT and WHERE — Pick Columns, Filter Rows

	What it does	Example on sales.orders
= > < >=	Compare a column to a value	WHERE order_total >= 100
BETWEEN / IN	Match a range or a short list	WHERE order_date BETWEEN '2026-09-01' AND '2026-09-30'
LIKE	Match a text pattern	WHERE email LIKE '%@gmail.com'
IS NULL	Find rows with a missing value	WHERE shipped_at IS NULL

```
-- September shipped orders over 500 QR
SELECT order_id, customer_id, order_total
FROM sales.orders
WHERE order_date >= '2026-09-01'
      AND status = 'shipped'
      AND order_total > 500;
```

⚠ Common traps

- NULL means unknown: status = NULL matches nothing. Use IS NULL.
- SELECT * breaks when columns change — name the columns you need.

SQL Runs in a Logical Order — Not the Order You Write

Logical execution order

1 FROM — pick the tables



2 WHERE — filter rows



3 GROUP BY — build groups



4 HAVING — filter groups



5 SELECT — choose columns



6 ORDER BY — sort output

The order you write it

```
SELECT country, COUNT(*) AS orders
FROM sales.orders
WHERE order_date >= '2026-01-01'
GROUP BY country
HAVING COUNT(*) > 100
ORDER BY orders DESC
LIMIT 10;
```

Why it matters: WHERE filters rows before grouping; HAVING filters after

Joins Combine Tables

	What you get	Use it when
INNER	Only rows matching in both tables	Every order must have a valid customer
LEFT	All left rows; NULLs where no match	Keep all customers, even those with no orders
RIGHT	All right rows; NULLs where no match	Rarely used — rewrite as LEFT for readability

```
SELECT c.customer_name,  
       COUNT(o.order_id) AS orders  
FROM sales.customers AS c  
LEFT JOIN sales.orders AS o  
      ON o.customer_id = c.customer_id  
GROUP BY c.customer_name;
```

Common trap

- A LEFT JOIN plus a WHERE filter on the right table silently becomes an INNER JOIN.
- Put that condition in the ON clause instead.



Aggregation Turns Raw Rows Into Business Metrics

BEFORE — raw order rows

order_id	country	amount
1001	Qatar	120.00
1002	Qatar	80.00
1003	UAE	200.00
1004	UAE	50.00
1005	Qatar	45.00

The query

```
SELECT country,  
       COUNT(*) AS orders,  
       SUM(amount) AS revenue,  
       AVG(amount) AS avg_order  
FROM sales.orders  
GROUP BY country  
HAVING SUM(amount) > 100  
ORDER BY revenue DESC;
```

AFTER — one row per group

country	orders	revenue	avg_order
Qatar	3	245.00	81.67
UAE	2	250.00	125.00



Important Notes

- COUNT(*) counts rows; COUNT(col) skips NULLs.
- Every non-aggregated SELECT column must appear in GROUP BY.
- HAVING filters groups; WHERE filters rows before grouping.



CTEs Turn Nested Subqueries Into Readable Steps

BEFORE — nested subquery

```
SELECT customer_id, COUNT(*) AS cnt
FROM (
  SELECT *
  FROM sales.orders
  WHERE order_date >=
    CURRENT_DATE - INTERVAL '30' DAY
) AS t
GROUP BY customer_id;
```

AFTER — named CTE

```
WITH recent_orders AS (
  SELECT *
  FROM sales.orders
  WHERE order_date >=
    CURRENT_DATE - INTERVAL '30' DAY
)
SELECT customer_id, COUNT(*) AS cnt
FROM recent_orders
GROUP BY customer_id;
```



Readable each step gets a name



Reusable reference a CTE more than once



Debuggable run one step at a time



Key idea

A Common Table Expression (CTE) is a name for a result set, not a stored table — it disappears when the query ends.



INSERT, UPDATE, DELETE — Changing Data Safely

	What it does	Example on silver.orders
INSERT	Add new rows to a table	<code>INSERT INTO silver.orders SELECT ... FROM bronze.orders</code>
UPDATE	Change values in rows that already exist	<code>UPDATE silver.orders SET status = 'delivered'</code>
DELETE	Remove the rows that match a condition	<code>DELETE FROM silver.orders WHERE order_id = 1042</code>
MERGE	Update the row if it exists, insert it if it does not	<code>MERGE INTO silver.orders USING bronze.orders ON order_id ...</code>

```
-- INSERT: append one day of orders
INSERT INTO silver.orders
  SELECT * FROM bronze.orders
  WHERE order_date = '2026-09-11';

-- MERGE: the rerun-safe version of that load
MERGE INTO silver.orders AS s
  USING bronze.orders AS b ON s.order_id = b.order_id
  WHEN MATCHED THEN UPDATE SET s.status = b.status
  WHEN NOT MATCHED THEN INSERT (order_id, status)
    VALUES (b.order_id, b.status);
```



Common traps

- **Prevent full-table updates:** A missing clause can update every row. Test the WHERE clause with a SELECT before running an UPDATE or DELETE.
- **Ensure safe reruns (idempotent loads):** Use MERGE with a reliable key to update matches, insert new rows, and prevent duplicates.



SQL Best Practices

	Why it matters	Do this
Name your columns	A new upstream column silently changes what SELECT * returns	SELECT order_id, status -- not SELECT *
Filter early	Rows dropped before a join never have to be moved or matched	WHERE order_date >= '2026-09-01'
Qualify and alias	Bare table names break when the same name exists in two schemas	FROM silver.orders AS o
Handle NULLs on purpose	Unknown values quietly drop rows and skew every average	COALESCE(discount, 0); IS NOT NULL

```
-- readable, filtered early, NULL-safe
SELECT o.order_id,
       c.customer_name,
       COALESCE(o.discount, 0) AS discount
FROM   silver.orders      AS o
JOIN   silver.customers  AS c
ON     c.customer_id = o.customer_id
WHERE  o.order_date >= '2026-09-01'
AND    o.status = 'shipped';
```

⚠ Common traps

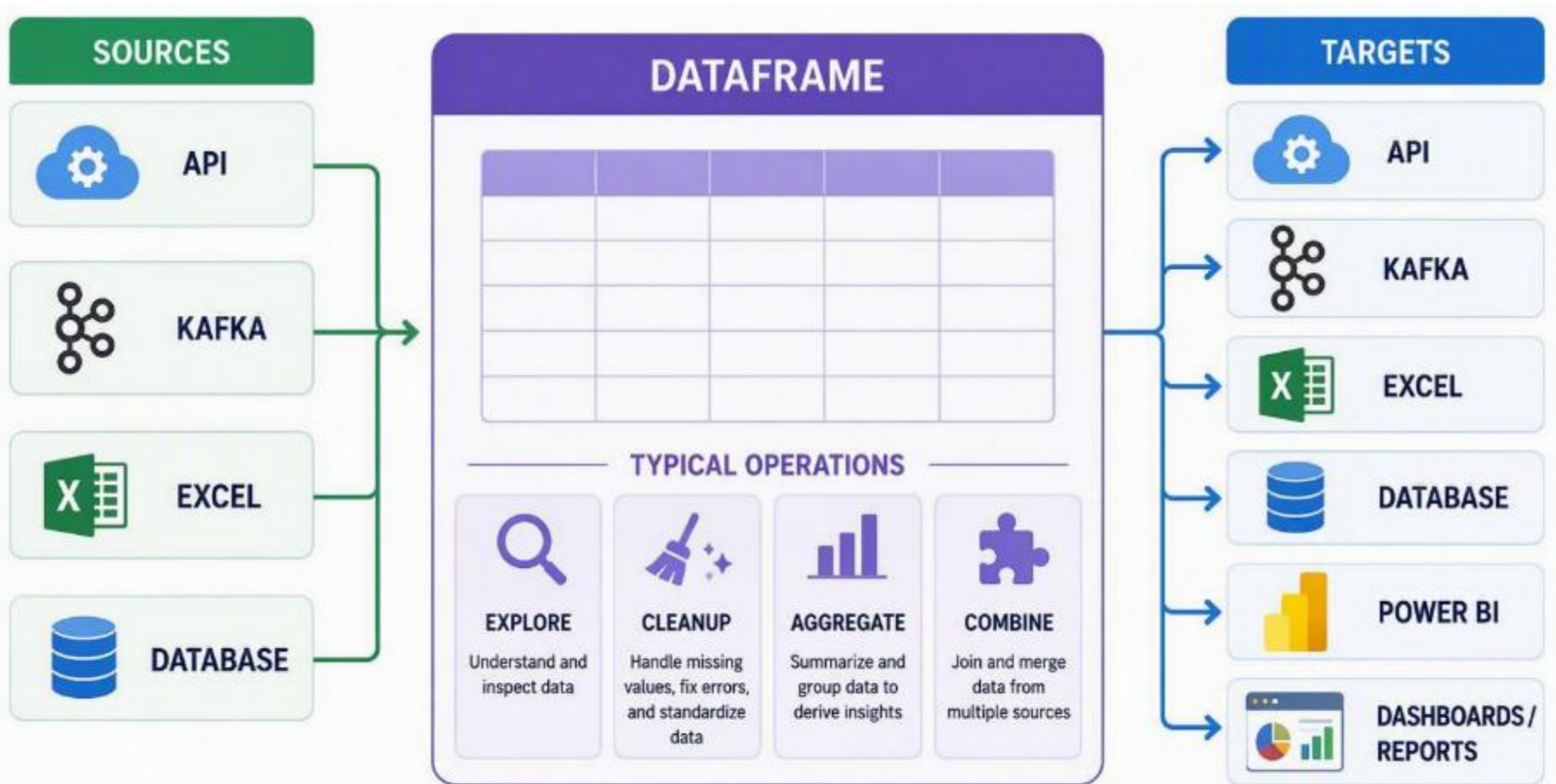
WHERE vs HAVING: filter rows in WHERE; use HAVING only for aggregates like COUNT(*) > 5.

SELECT * in a pipeline: one new upstream column can silently break tomorrow's load.



Data Processing using DataFrame

DataFrame = a two-dimensional, labeled tabular structure with columns, rows, types/schema



ONE STRUCTURE. MANY PURPOSES. DataFrame connects diverse sources to multiple targets through consistent operations—powering both data engineering and analytics.

Pandas, Polars, PySpark: Same DataFrame, Different Engine

	 pandas	 Polars	 PySpark
Best for	Exploration, small ad-hoc analysis	Mid-size data on one machine, CPU-heavy work	Large-scale ETL pipelines, complex joins, streaming
Sweet spot	< 5M rows, < 500 MB files	5M–50M rows, multi-GB CSV / Parquet	Datasets too large for one machine; 100M+ rows
Strength	Simple, flexible, huge ecosystem	Multi-core processing, lazy evaluation, and a columnar memory format for fast processing	Distributed processing, scalable transformations
Execution	Eager — runs each step right away	Lazy — optimizes the whole query first	Lazy — plans first, then runs it efficiently
Limit	Single-threaded, memory-bound, eager only	Single machine — not distributed	Cluster overhead on small jobs; costly infrastructure to set up

Rule of thumb: start on the smallest engine that fits your data; move up only when size or governance forces it.

On 10M rows: pandas ~40s • Polars ~5s • Spark 8–12s locally, but scales past 100M rows.



Read & Filter - The SELECT and WHERE of DataFrame

	SQL clause	Pandas equivalent on df
SELECT cols	Pick a subset of columns	<code>df[["order_id", "order_total"]]</code>
WHERE cond	Keep rows that match a mask	<code>df[df.order_total > 500]</code>
IN (list)	Match any value in a list	<code>df[df.status.isin(["shipped", "paid"])]</code>
SELECT + WHERE	Filter rows and columns at once	<code>df.loc[df.order_total > 500, ["order_id", "order_total"]]</code>

```
# September shipped orders over 500 QAR
df = pd.read_parquet("bronze/orders")
cols = ["order_id", "customer_id",
"order_total"]
m = (df.status == "shipped") & \
    (df.order_total > 500) & \
    (df.order_date >= "2026-09-01")
out = df.loc[m, cols]
```

⚠ Common traps

- **Combining filters:** Use **&** and **|**, with each condition in parentheses.

```
df[(df["sales"] > 1000) &
(df["region"] == "Qatar")]
```

- Use `.loc[mask, columns]` for safe filtering and assignment.

```
df.loc[(df["sales"] > 1000) & (df["region"]
== "Qatar"), "priority"] = "High"
```



pd.merge — Joining DataFrames, the SQL JOIN

	SQL JOIN	pd.merge equivalent
INNER JOIN	Keep only matching keys	how="inner" (default)
LEFT JOIN	All orders, matched customers	how="left"
RIGHT JOIN	All customers, matched orders	how="right"
FULL OUTER JOIN	Keep every row from both	how="outer"

```
# Attach the customer to each order
orders = pd.read_parquet("bronze/orders")
customers =
pd.read_parquet("bronze/customers")
df = pd.merge(orders, customers,
              on="customer_id",
              how="left")
```

⚠ Common traps

- One-to-many keys multiply rows: if a customer_id repeats, each order matches it more than once, so rows quietly duplicate. Add validate="m:1" to catch it.
- Same-named columns clash — e.g. two total columns. Keep both with suffixes=("_ord", "_cust"), giving total_ord and total_cust.



groupby & agg — The GROUP BY of DataFrames

	SQL	Pandas equivalent
GROUP BY customer_id	Build one group per customer	<code>df.groupby("customer_id")</code>
SUM(order_total)	Total revenue per group	<code>.agg(revenue=("order_total", "sum"))</code>
COUNT(*)	Number of orders per group	<code>.agg(n_orders=("order_id", "count"))</code>
HAVING SUM(...) > 1000	Filter groups after aggregating	<code>g[g.revenue > 1000]</code>

```
# Revenue per customer for September
g = (df.groupby("customer_id")
     .agg(
         revenue=("order_total", "sum"),
         n_orders=("order_id", "count"))
     .reset_index())
```

⚠ Common traps

- HAVING has no direct method: aggregate first, then filter the result with a boolean mask.
- agg skips NaN by default, so sum and count ignore missing values



Transform & Upsert — Writing bronze into silver

	What it does	Pandas on df / silver
Add column	Derive a new column from existing ones	<code>df.assign(with_tax=df.order_total * 1.05)</code>
Update rows	Change values where a mask is true	<code>df.loc[df.order_id == 1042, "status"] = "delivered"</code>
Remove	Drop columns (or rows) you no longer need	<code>df.drop(columns=["_ingest_tmp"])</code>
Upsert	Update matches and append new rows, rerun-safe	<code>pd.concat([silver, df]).drop_duplicates("order_id", keep="last")</code>

```
# Add, update and remove columns/rows
df = df.assign(with_tax=df.order_total * 1.05)
df.loc[df.order_id == 1042, "status"] = "delivered"
df = df.drop(columns=["_ingest_tmp"])

# Upsert: the rerun-safe write into silver
silver = pd.read_parquet("silver/orders")
combined = pd.concat([silver, df])
silver = combined.drop_duplicates("order_id", keep="last")
silver.to_parquet("silver/orders")
```



Common traps

- **assign and drop return copies:** they do not change df in place, so reassign the result (df = df.assign(...)) or your edit is lost.
- **Make the upsert idempotent:** concat then drop_duplicates on the key with keep=last, so a rerun overwrites instead of doubling rows.



Clean & Missing Data — Preparing bronze for silver

	Cleaning task	Pandas method on df
Dedupe	Drop repeated order_id rows	<code>df.drop_duplicates("order_id")</code>
Drop missing	Remove rows with no key	<code>df.dropna(subset=["order_id"])</code>
Fill missing	Replace NaN with a default	<code>df.status.fillna("unknown")</code>
Cast types	Enforce the silver schema	<code>df.order_total.astype("float64")</code>

```
# Tidy bronze orders before the load
df = df.drop_duplicates("order_id")
df = df.dropna(subset=["order_id"])
df["status"] = df.status.fillna("unknown")
df["order_total"] =
df.order_total.astype("float64")
df["order_date"] =
pd.to_datetime(df.order_date)
```

⚠ Common traps

- `fillna(0)` on numbers hides missing values and skews sums and averages — fill deliberately.
- `astype("int64")` fails if NaN remains; use the nullable `"Int64"` dtype instead.

Process columns as a pipeline - not row by row

Pipeline = chain of operations on orders: filter → clean → derive → group → summarize → sort.

```
summary = (orders
    .loc[orders["status"] == "Complete"]
    .dropna(subset=["amount"])
    .drop_duplicates("order_id")
    .assign(net_amount=lambda d:
            d["amount"] * 0.95)
    .groupby("channel", as_index=False)
    .agg(orders=("order_id", "nunique"),
        revenue=("net_amount", "sum"))
    .sort_values("revenue", ascending=False)
)
```

Read the chain

- 1 Keep complete orders
- 2 Handle missing + duplicates
- 3 Create net_amount
- 4 Group by channel
- 5 Count orders + sum revenue
- 6 Put highest revenue first

Resources

DataFrames & engines

[pandas](#) - Official user guide and API reference

[Polars](#) - Fast multi-threaded DataFrames

[PySpark](#) - Distributed DataFrames

SQL

[SQL Tutorial](#) - Interactive SQL, basics to advanced

Lakehouse

Get Started with [Databricks for Data Engineering](#)

Get Started with [DuckLake](#)